Who Wrote It? Identifying LLMs from Their Responses

Midterm Project report

Pradyumna Vallabhajosyula

Introduction-

I did this project individually; the project investigates whether text was generated by different large language models containing patterns that can be used to identify its source. I implemented a Convolutional Neural Network (CNN) and a Long Short-Term Memory network (LSTM), a type of recurrent neural network. I evaluated both models using the user's prompt alone, the model's response alone, and the prompt and response together. I focused on research questions 1 and 2, whether the responses can be distinguished from each other and whether including the prompts can improve classification.

Data selection-

The data I used came from publicly available sets on OpenBMB, I used the UltraFeedback dataset and downloaded it through the hugging face library The dataset includes prompts/responses from multiple types of LLMs. I selected GPT-4, Llama-2-70B-chat, and Falcon-40B-instruct to represent three different LLM families. I used the existing model labels rather than manually labeling responses. Preparing the dataset involved several steps before training: selecting the three model families, checking the available response counts, removing empty and duplicate entries, grouping matching prompts, and balancing the classes within each split.

The final data set had 9000 examples, that's 3000 each, each example included the model's name, user prompt, and generated response, I did not use the dataset's ratings, feedback, or other annotations as classifier inputs. For each model, I used 2,100 examples for training, 450 for validation, and 450 for testing. Before sampling these balanced subsets, I divided unique prompts into training, validation, and testing groups. I normalized prompts by converting them to lowercase and collapsing whitespace for

grouping purposes. This prevented responses to the same normalized prompt from appearing across different splits.

Preprocessing-

The text was tokenized, that means I made it all lowercase and split it into words and punctuation, this allowed the models to use some of the formatting information to make a decision as well so I thought it would be worth it to keep. I constructed a shared vocabulary of 20,000 tokens using only the training prompts and responses. Tokens outside this vocabulary were represented by an unknown-token ID.

For input-only classification, I kept up to 128 tokens from the prompt. For output-only classification, I kept up to 256 tokens from their response. The combined condition used both the limits, with a separator between the prompt and the response. This meant that adding the prompt did not reduce the amount of response text that was available. I also padded sequences within each batch and handled that padding in both models so it would not affect their representations.

Models and Training-

I used a CNN and an LSTM to compare two different ways of processing the text. Both started with a trainable embedding layer that represented each token as a vector with 128 dimensions.

The CNN used filters of different sizes, covering 3, 4 and 5 tokens at a time, it used 65 filters of each size, the filters allow it to capture small patterns in the text. Max pooling selected the strongest features, which were then combined and passed to the classification layer.

I trained both models for five epochs in each of the three input conditions. I used Adam with a learning rate of 0.001, a batch size of 64, cross-entropy loss, and dropout of 0.3. I also clipped gradients to a maximum norm of 1.0 and used a random seed of 42. The LSTM processed the tokens in order, carrying information through its hidden state. It had one recurrent layer with a hidden size of 128. I used its final hidden state to predict which model generated the response. I chose an embedding dimension and LSTM

hidden size of 128 to give the models enough capacity to learn text patterns while keeping training manageable. I did not tune these values.

One difficulty was that the input-only models continued to reduce their training loss without a clear improvement in validation performance. This suggested they were learning patterns in the training prompts that did not carry over to unseen prompts. For each experiment, I selected the checkpoint with the highest validation macro F1 rather than automatically using the final epoch, then evaluated that checkpoint on the test set.

Results –

I measured performance using accuracy and macro F1. Accuracy shows how often the prediction was correct, while macro F1 gives equal weight to the performance on each class. Since the test set was balanced across three classes, chance accuracy was 33.33%.

Model	Text used	Test accuracy	Test macro F1
CNN	Input only	32.00%	0.3197
CNN	Output only	78.15%	0.7815
CNN	Input + output	74.74%	0.7457
LSTM	Input only	33.19%	0.3280
LSTM	Output only	65.33%	0.6521
LSTM	Input + output	64.74%	0.6469

RQ1: Can we identify which LLM generated a response?

Yes, we can.

Both classifiers could identify the generating LLM above chance using its response, with the CNN performing better than the LSTM. Using only the output, the CNN reached 78.15% accuracy and LSTM reached 65.33%, compared with a chance baseline of

33.33%. This suggests that the responses contained some patterns that helped distinguish the three selected models.

The CNN did better by almost 13% (12.82%), one explanation for that could be that the short patterns make text more recognizable. Short patterns, such as repeated phrases or punctuation sequences, were useful and suited the CNN's filters.

RQ2: Does the user's prompt help identify the LLM?

No, as far as I can tell it does not.

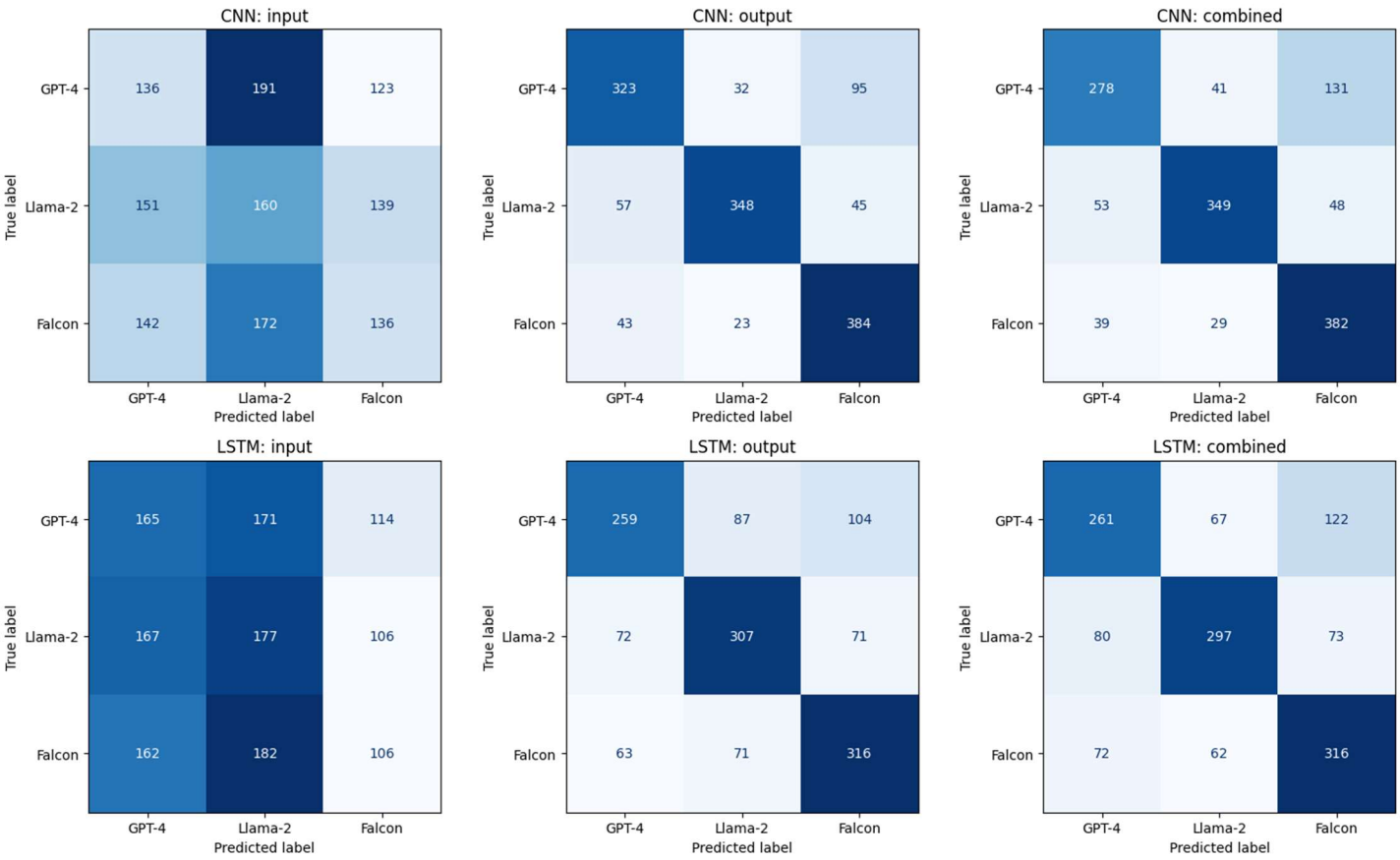
The user's prompt did not improve classification in this experiment, and the prompt alone performed close to (and slightly worse than) chance. Input-only accuracy was 32.00% for the CNN and 33.19% for the LSTM. This makes sense because the models answered prompts that were taken from a shared collection, so there's no reason for them to be measurably different.

Adding the prompt and the response together reduced CNN accuracy from 78.15% to 74.74% and LSTM accuracy from 65.33% to 64.74%. The prompt probably added unnecessary context that threw off the recognition. This is still much higher than chance but worse than without. I can't outright say that it is completely the reason but I have no reason to believe it helps.

Limitations-

Using only one example of each LLM "family" means I cannot say whether this would work on other versions of GPT, Llama or Falcon. UltraFeedback also includes different generation instructions, which could have affected the responses. Some of the patterns learned by the classifiers may therefore reflect those instructions as well as differences between the models. Longer prompts and responses were cut off and converting everything to lowercase removed capitalization patterns. I kept matching normalized prompts out of different splits, but this did not rule out overlap between prompts with similar meanings. Finally, I used one random seed for each experiment and did not test performance across separate domains or investigate which specific features were responsible for classification.

I made a set confusion matrices for clarity on the effectiveness -



Dataset Reference-

OpenBMB. *UltraFeedback*. https://huggingface.co/datasets/openbmb/UltraFeedback